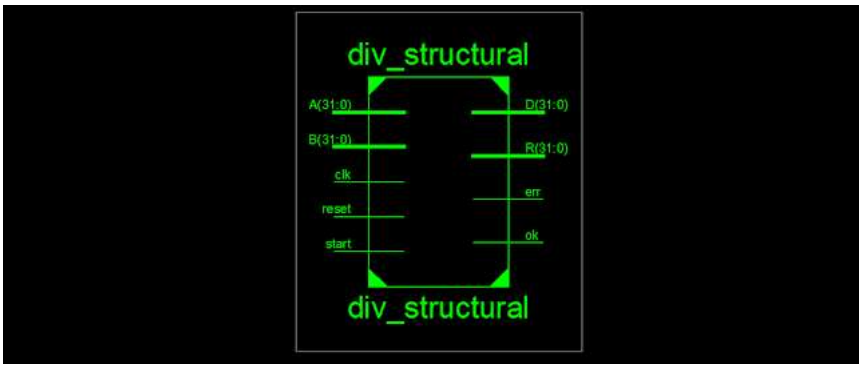
32-bit Unsigned Divider in Verilog

In this project, a 32-bit unsigned divider is implemented in Verilog using both structural and behavioral models. The Verilog code for the divider is synthesizable and can be implemented on FPGA.



Structural Verilog code for 32-bit unsigned divider:

```
`timescale 1ns / 1ns
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 32-bit divider
// Verilog code for divider using structural modelling
module div_structural(
  input      clk,
      input                    reset,
  input      start,
  input [31:0]  A,
  input [31:0]  B,
  output [31:0]  D,
  output [31:0]  R,
  output     ok ,   // =1 when ready to get the result
      output err
  );
  wire       active;   // True if the divider is running
  wire [4:0]    cycle;   // Number of cycles to go
      wire [4:0] cycle_d;
      wire [31:0]denom_d,work_d,result_d;
  wire [31:0]   result;   // Begin with A, end with D
  wire [31:0]   denom;   // B
  wire [31:0]   work;    // Running R
      wire start_n,clr,active_d;
  // Calculate the current digit
  wire [32:0]   sub = { work[30:0], result[31] } - denom;
      assign err = !B;
  // Send the results to our master
  assign D = result;
  assign R = work;
  assign ok = ~active;
      not n1(start_n,start);
      or o1(clr,reset,start_n);
      dff u1(.q(active), .d(active_d), .reset(clr), .clk(clk));
      assign active_d = active ? ((cycle==0)?1'b0:active): 1'b1;
      register_5 cycle_reg(cycle,cycle_d,1'b1,clr,clk);
      assign cycle_d = active?(cycle-5'd1):5'd31;
      register reg32_denom(denom,denom_d,1'b1,clr,clk);
      assign denom_d = active?denom:B;
      register reg32_work(work,work_d,1'b1,clr,clk);
      assign work_d = active?((sub[32]==0)?sub[31:0]:{work[30:0], result[31]}):0;
      register reg32_result(result,result_d,1'b1,clr,clk);
      assign result_d = active==1 ?((sub[32]==0)?{result[30:0], 1'b1}:{result[30:
```

```
endmodule
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 32-bit divider
// Verilog code for divider using structural modelling module
RegBit(BitOut, BitData, WriteEn,reset, clk);
output BitOut; // 1 bit of register
input BitData, WriteEn;
input reset,clk;
wire d,f1, f2; // input of D Flip-Flop
wire reset;
and #(50) U1(f1, BitOut, (~WriteEn));
and #(50) U2(f2, BitData, WriteEn);
or #(50) U3(d, f1, f2);
dff DFF0(BitOut, d, reset, clk);
endmodule
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 32-bit divider
// Verilog code for divider using structural modelling
module register_5(RegOut,RegIn,WriteEn,reset,clk);
output [4:0] RegOut;
input [4:0] RegIn;
input WriteEn,reset, clk;
RegBit     bit4 (RegOut[4], RegIn[4], WriteEn,reset,clk);
RegBit     bit3 (RegOut[3], RegIn[3], WriteEn,reset,clk);
RegBit     bit2 (RegOut[2], RegIn[2], WriteEn,reset,clk);
RegBit     bit1 (RegOut[1], RegIn[1], WriteEn,reset,clk);
RegBit     bit0 (RegOut[0], RegIn[0], WriteEn,reset,clk);
endmodule
// 32-bit register
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 32-bit divider
// Verilog code for divider using structural modelling
module register(RegOut,RegIn,WriteEn,reset,clk);
output [31:0] RegOut;
input [31:0] RegIn;
input WriteEn,reset, clk;
RegBit     bit31(RegOut[31],RegIn[31],WriteEn,reset,clk);
RegBit     bit30(RegOut[30],RegIn[30],WriteEn,reset,clk);
RegBit     bit29(RegOut[29],RegIn[29],WriteEn,reset,clk);
RegBit     bit28(RegOut[28],RegIn[28],WriteEn,reset,clk);
RegBit     bit27(RegOut[27],RegIn[27],WriteEn,reset,clk);
RegBit     bit26(RegOut[26],RegIn[26],WriteEn,reset,clk);
RegBit     bit25(RegOut[25],RegIn[25],WriteEn,reset,clk);
RegBit     bit24(RegOut[24],RegIn[24],WriteEn,reset,clk);
RegBit     bit23(RegOut[23],RegIn[23],WriteEn,reset,clk);
RegBit     bit22(RegOut[22],RegIn[22],WriteEn,reset,clk);
RegBit     bit21(RegOut[21],RegIn[21],WriteEn,reset,clk);
RegBit     bit20(RegOut[20],RegIn[20],WriteEn,reset,clk);
RegBit     bit19(RegOut[19],RegIn[19],WriteEn,reset,clk);
RegBit     bit18(RegOut[18],RegIn[18],WriteEn,reset,clk);
RegBit     bit17(RegOut[17],RegIn[17],WriteEn,reset,clk);
RegBit     bit16(RegOut[16],RegIn[16],WriteEn,reset,clk);
RegBit     bit15(RegOut[15],RegIn[15],WriteEn,reset,clk);
RegBit     bit14(RegOut[14],RegIn[14],WriteEn,reset,clk);
RegBit     bit13(RegOut[13],RegIn[13],WriteEn,reset,clk);
RegBit     bit12(RegOut[12],RegIn[12],WriteEn,reset,clk);
RegBit     bit11(RegOut[11],RegIn[11],WriteEn,reset,clk);
RegBit     bit10(RegOut[10],RegIn[10],WriteEn,reset,clk);
RegBit     bit9 (RegOut[9], RegIn[9], WriteEn,reset,clk);
RegBit     bit8 (RegOut[8], RegIn[8], WriteEn,reset,clk);
RegBit     bit7 (RegOut[7], RegIn[7], WriteEn,reset,clk);
RegBit     bit6 (RegOut[6], RegIn[6], WriteEn,reset,clk);
RegBit     bit5 (RegOut[5], RegIn[5], WriteEn,reset,clk);
```

```
RegBit     bit4 (RegOut[4], RegIn[4], WriteEn,reset,clk);
RegBit     bit3 (RegOut[3], RegIn[3], WriteEn,reset,clk);
RegBit     bit2 (RegOut[2], RegIn[2], WriteEn,reset,clk);
RegBit     bit1 (RegOut[1], RegIn[1], WriteEn,reset,clk);
RegBit     bit0 (RegOut[0], RegIn[0], WriteEn,reset,clk);
endmodule
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
module dff (q, d, reset, clk);
output q;
input d, reset, clk;
reg q; // Indicate that q is stateholding
always @(posedge clk or posedge reset)
if (reset)
q = 0;     // On reset, set to 0
else
q = d; // Otherwise out = d
endmodule
```

Behavioral Verilog code for 32-bit unsigned divider:

```
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 32-bit divider
// Verilog code for divider using behavioral modelling
module Divide(
  input      clk,
      input                    reset,
  input      start,
  input [31:0]  A,
  input [31:0]  B,
  output [31:0]  D,
  output [31:0]  R,
  output     ok ,   // =1 when ready to get the result
      output err
  );
  reg       active;   // True if the divider is running
  reg [4:0]    cycle;   // Number of cycles to go
  reg [31:0]   result;   // Begin with A, end with D
  reg [31:0]   denom;   // B
  reg [31:0]   work;    // Running R
  // Calculate the current digit
  wire [32:0]   sub = { work[30:0], result[31] } - denom;
      assign err = !B;
  // Send the results to our master
  assign D = result;
  assign R = work;
  assign ok = ~active;
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
  // The state machine
  always @(posedge clk,posedge reset) begin
    if (reset) begin
      active <= 0;
      cycle <= 0;
      result <= 0;
      denom <= 0;
      work <= 0;
    end
    else if(start) begin
      if (active) begin
        // Run an iteration of the divide.
        if (sub[32] == 0) begin
          work <= sub[31:0];
          result <= {result[30:0], 1'b1};
        end
        else begin
          work <= {work[30:0], result[31]};
          result <= {result[30:0], 1'b0};
        end
        if (cycle == 0) begin
          active <= 0;
        end
```

```
        cycle <= cycle - 5'd1;
      end
      else begin
        // Set up for an unsigned divide.
        cycle <= 5'd31;
        result <= A;
        denom <= B;
        work <= 32'b0;
        active <= 1;
      end
    end
  end
endmodule
```

Verilog testbench code for Structural divider:

```
`timescale 1ns / 1ns
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for 32-bit divider
// Testbench Verilog code for divider using structural modelling
module tb_div_structural;
     // Inputs
     reg clock;
     reg reset;
     reg start;
     reg [31:0] A;
     reg [31:0] B;
     // Outputs
     wire [31:0] D;
     wire [31:0] R;
     wire ok;
     wire err;
     // Instantiate the Unit Under Test (UUT)
     div_structural uut (
          .clk(clock),
          .start(start),
          .reset(reset),
          .A(A),
          .B(B),
          .D(D),
          .R(R),
          .ok(ok),
          .err(err)
     );
     initial begin
           clock = 0;
           forever #50 clock = ~clock;
     end
     initial begin
          // Initialize Inputs
          start = 0;
          A = 32'd1023;
          B = 32'd50;
          reset=1;
          // Wait 100 ns for global reset to finish
          #1000;
          reset=0;
    start = 1;
          #5000;
          $finish;
     end
endmodule
```

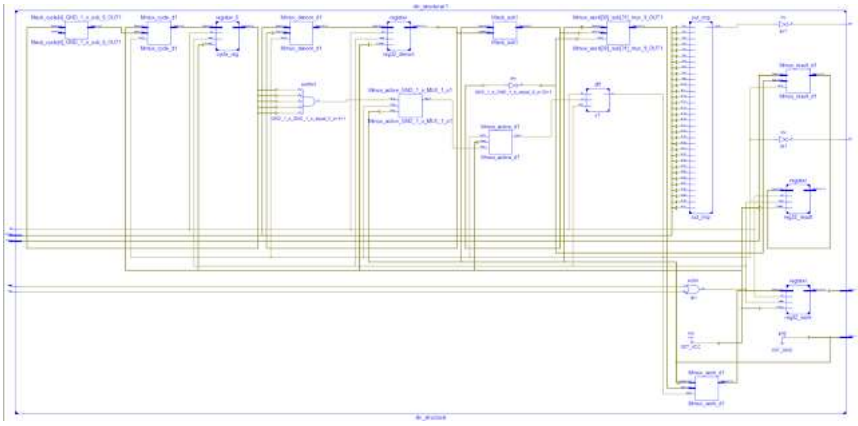
Verilog testbench code for Behavioral divider:

```
`timescale 1ns / 1ns
 // fpga4student.com FPGA projects, Verilog projects, VHDL projects
```

```
// Verilog project: Verilog code for 32-bit divider
// Testbench Verilog code for divider using behavioral modelling
module tb_divider;
     // Inputs
     reg clock;
     reg reset;
     reg start;
     reg [31:0] A;
     reg [31:0] B;
     // Outputs
     wire [31:0] D;
     wire [31:0] R;
     wire ok;
     wire err;
     // Instantiate the Unit Under Test (UUT)
     Divide uut (
          .clk(clock),
          .start(start),
          .reset(reset),
          .A(A),
          .B(B),
          .D(D),
          .R(R),
          .ok(ok),
          .err(err)
     );
     initial begin
           clock = 0;
           forever #50 clock = ~clock;
     end
     initial begin
          // Initialize Inputs
          start = 0;
          A = 32'd1023;
          B = 32'd50;
          reset=1;
          // Wait 100 ns for global reset to finish
          #1000;
          reset=0;
    start = 1;
          #5000;
          $finish;
     end
endmodule
```

Simulation waveform for the divider:

Synthesized RTL Schematic for the divider:



Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. **How to generate a clock enable signal in Verilog**
34. **Verilog code for PWM Generator**
35. **Verilog coding vs Software Programming**
36. **Verilog code for Moore FSM Sequence Detector**

37. **Verilog code for 7-segment display controller on Basys 3 FPGA**

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.